

SAPPHIRE PROGRAMMING LANGUAGE

Formal Language Specification, AST Grammar, Standard Library Architecture & Automation Manual

Version 1.0.0 (Core Architecture Spec) | Sapphire Language Architecture Team

1. Executive Specification & Grammar Architecture

This document defines the formal language specification and runtime standard library for the **Sapphire Programming Language**. Sapphire is parsed using a recursive-descent syntax tree generator and executed via an AST visitor runtime with lexically scoped environments.

```
Program      ::= Statement*
Statement    ::= VarDecl | AssignStmt | FnDecl | ReturnStmt | IfStmt
               | WhileStmt | ForStmt | ParallelStmt | ExprStmt
VarDecl      ::= "let" IDENTIFIER (":" Type)? ("=" Expression)? ";"
FnDecl       ::= "fn" IDENTIFIER "(" ParamList? ")" ("->" Type)? Block
ParallelStmt ::= "parallel" Block
PipeExpr     ::= LogicalOr ( ">" LogicalOr )*
MemberAccess ::= Primary ( "." IDENTIFIER | "[" Expression "]" | "(" ArgList? ")" )*
```

2. Lexical Tokens, Keywords & Operators

Category	Token Literals	Semantic Description
Keywords	let, fn, return, if, else, while, for, in, parallel, break, continue	Reserved language grammar keywords.
Operators	+, -, *, /, %, ==, !=, <, >, <=, >=, and, or, not, >	Arithmetic, logical, comparison, and stream piping.
Literals	42, 3.1415, "hello", true, false, null, [1, 2], {"k": "v"}	Numbers, strings, booleans, null, arrays, maps.

3. Runtime Execution & Scoping Rules

Sapphire employs a hierarchical lexically scoped environment model. Inner blocks inherit bindings from enclosing parent environments. Green fiber concurrency within parallel { ... } blocks encapsulates memory isolation while allowing structured rendezvous synchronization.

4. Deep Learning Standard Library (ml)

Function	Signature	Description
<code>ml.tensor</code>	<code>ml.tensor(data, dtype, device)</code>	Constructs N-dimensional tensor.
<code>ml.autograd.tape</code>	<code>ml.autograd.tape()</code>	Context manager for automatic differentiation.
<code>ml.model.mlp</code>	<code>ml.model.mlp(layers, act)</code>	Builds multi-layer perceptron neural model.
<code>ml.train.fit</code>	<code>ml.train.fit(model, ds, loss, opt, ...)</code>	Trains model across N data-parallel workers.
<code>ml.gpu.info</code>	<code>ml.gpu.info() -> Array[Map]</code>	Queries NVIDIA CUDA GPU telemetry.
<code>ml.kernel.conv2d</code>	<code>ml.kernel.conv2d(img, kernel)</code>	Applies 2D convolution kernel filter.

5. Autonomous Agent Standard Library (agent)

Function	Signature	Description
<code>agent.memory.remember</code>	<code>remember(key, val, embedding=None)</code>	Saves knowledge into vector memory store.
<code>agent.memory.recall</code>	<code>recall(key) -> Any</code>	Recalls value from memory store.
<code>agent.planning.create_plan</code>	<code>create_plan(goal: str) -> Plan</code>	Generates 4-step DAG action plan.
<code>agent.tools.register</code>	<code>register(name, desc, fn)</code>	Registers runtime executable tool.
<code>agent.autonomy.run_loop</code>	<code>run_loop(goal, max_steps)</code>	Executes autonomous self-correcting agent loop.

6. System Automation Standard Library (os, fs, gui, http)

API	Signature	Description
<code>os.system_info</code>	<code>os.system_info() -> Map</code>	Returns CPU, RAM, and Disk metrics.
<code>os.notify</code>	<code>os.notify(title, msg) -> Void</code>	Displays native OS desktop notification toast.
<code>fs.read / fs.write</code>	<code>fs.read(path) / fs.write(path, data)</code>	Reads and writes filesystem files.
<code>gui.click / gui.alert</code>	<code>gui.click(x, y) / gui.alert(msg, title)</code>	Simulates mouse clicks and modal popups.
<code>http.get / http.post</code>	<code>http.get(url) / http.post(url, json)</code>	Performs REST network requests.
<code>scheduler.interval</code>	<code>scheduler.interval(secs, fn)</code>	Schedules recurring background execution.